

Engine Orchestrator

Objective

Create a deterministic **Engine Orchestrator** that runs all EnterprateAI engines in the correct sequence from a single input and outputs a **unified structured response**.

It sequences:

1. Viability Engine
2. Survival Engine
3. Stability Engine
4. Growth Engine
5. Master Aggregator
6. Fragility Index
7. Scenario Engine / ASI (optional, if scenario changes are provided)

It ensures:

- deterministic execution
- proper input/output mapping between engines
- consolidated risk, recommendations, scores
- traceability for AI narratives and PDF reports

Recommended Folder

/src/lib/engines/orchestrator

Suggested structure:

```
/src/lib/engines/orchestrator
├── types.ts
├── orchestrator.ts
├── orchestrator.test.ts
└── index.ts
```

1. Input Contract

```
export interface EngineOrchestratorInput {
  businessId: string;

  financialData: ViabilityEngineInput["costs"] &
  ViabilityEngineInput["revenue"] & SurvivalEngineInput["financials"];

  commercialData: ViabilityEngineInput["demand"] &
  ViabilityEngineInput["capacity"];

  operationalData: StabilityEngineInput["operations"] &
  StabilityEngineInput["capacity"];

  marketData: GrowthEngineInput;

  customerData: StabilityEngineInput["customers"];

  dependencies?: StabilityEngineInput["dependencies"];

  scenarioChanges?: ScenarioEngineInput["scenarioChanges"]; //
  optional

  metadata?: {
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
    rulesetVersion?: string;
    source?: "user" | "system" | "asi";
  };
}
```

2. Output Contract

```
export interface EngineOrchestratorOutput {
  businessId: string;
```

```
engines: {
  viability: ViabilityEngineOutput;
  survival: SurvivalEngineOutput;
  stability: StabilityEngineOutput;
  growth: GrowthEngineOutput;
  master: MasterAggregatorOutput;
  fragility: FragilityEngineOutput;
  scenario?: ScenarioEngineOutput; // if scenarioChanges provided
};

unified: {
  businessIntelligenceScore: number;
  classification: MasterAggregatorOutput["classification"];
  topRisks: RiskFlag[];
  topRecommendations: Recommendation[];
  interpretation: {
    headline: string;
    summary: string;
    strategicPriority: string;
    nextBestActions: string[];
  };
  confidence: Confidence;
};

trace: Record<string, unknown>;
}
```

3. Execution Sequence

1. Viability Engine

Input: financialData + commercialData

Output: ViabilityEngineOutput

2. Survival Engine

Input: financialData + historical cashflow

Output: SurvivalEngineOutput

3. **Stability Engine**

Input: operationalData + customerData + dependencies

Output: StabilityEngineOutput

4. **Growth Engine**

Input: marketData

Output: GrowthEngineOutput

5. **Master Aggregator**

Input: Outputs of 1–4

Output: MasterAggregatorOutput

6. **Fragility Index**

Input: riskFlags + engine scores + dependencies

Output: FragilityEngineOutput

7. **Scenario Engine / ASI** (*optional*)

Input: scenarioChanges + base state from steps 1–6

Output: ScenarioEngineOutput

4. Orchestrator Logic

```
async function runOrchestrator(input: EngineOrchestratorInput):  
  Promise<EngineOrchestratorOutput> {  
    // Step 1: Run Viability  
    const viability =  
      runViabilityEngine({ ...input.financialData, ...input.commercialData }  
    );  
  
    // Step 2: Run Survival  
    const survival = runSurvivalEngine({ ...input.financialData });  
  
    // Step 3: Run Stability  
    const stability =  
      runStabilityEngine({ ...input.operationalData, ...input.customerData,  
        dependencies: input.dependencies });  
  
    // Step 4: Run Growth  
    const growth = runGrowthEngine({ ...input.marketData });
```

```
// Step 5: Run Master Aggregator
const master = runMasterAggregator({ viability, survival, stability,
growth });

// Step 6: Run Fragility Engine
const fragility = runFragilityEngine({
  scores: master.scores,
  risks:
[...viability.riskFlags, ...survival.riskFlags, ...stability.riskFlags
, ...growth.riskFlags],
  dependencies: input.dependencies
});

// Step 7: Optional Scenario / ASI
let scenario;
if (input.scenarioChanges && input.scenarioChanges.length > 0) {
  scenario = runScenarioEngine({
    baseState: { viability, survival, stability, growth, fragility,
master },
    scenarioChanges: input.scenarioChanges
  });
}

// Step 8: Combine unified outputs
const unified = generateUnifiedOutput({ viability, survival,
stability, growth, master, fragility, scenario });

return {
  businessId: input.businessId,
  engines: { viability, survival, stability, growth, master,
fragility, scenario },
  unified,
  trace: {
    orchestrationSequence: ["viability", "survival", "stability",
"growth", "master", "fragility", scenario ? "scenario" : "none"]
  }
};
}
```

5. Unified Output Rules

- **businessIntelligenceScore:** from MasterAggregator
- **classification:** from MasterAggregator
- **topRisks:** consolidate riskFlags from all engines (consolidated)
- **topRecommendations:** consolidate and resolve conflicts
- **interpretation:** derived from MasterAggregator + Fragility + scenario simulation
- **confidence:** aggregate confidence of all engines

6. Unit Tests

- Run orchestrator with **normal inputs** → outputs all engines + unified object
- Run with **scenarioChanges** → includes scenario output
- Run with **missing optional inputs** → handles gracefully
- Run with **high-risk inputs** → risk flags propagate correctly
- Run with **low-confidence inputs** → confidence aggregation applied

7. Acceptance Criteria

- Orchestrator sequences all engines in order
- Handles optional scenario engine execution
- Consolidates outputs correctly into unified object
- Provides traceability for all steps
- No engine mutates live ontology
- Deterministic outputs for same inputs
- Unit tests pass

8. Codex Prompt

Build the EnterprateAI Engine Orchestrator in
`/src/lib/engines/orchestrator`.

It must be a pure TypeScript module that sequences engines:

1. Viability Engine
2. Survival Engine
3. Stability Engine
4. Growth Engine
5. Master Aggregator
6. Fragility Index
7. Scenario Engine / ASI (if `scenarioChanges` provided)

It should accept `EngineOrchestratorInput` and return
`EngineOrchestratorOutput`.

Steps:

- Run each engine deterministically
- Pass outputs of previous engines to Master Aggregator
- Run Fragility Engine with consolidated scores and risks
- If `scenarioChanges` exist, run Scenario Engine
- Consolidate unified business output (scores, classification, top risks, top recommendations, interpretation, confidence)
- Include trace object with orchestration sequence

Create:

- `types.ts`
- `orchestrator.ts`
- `orchestrator.test.ts`
- `index.ts`

Add unit tests for:

- normal execution
- scenario changes
- high-risk propagation
- low confidence

- missing optional data

Export the orchestrator from index.ts.